Level 2
Hackathon – Fundamentals of Web Development
Project: Online Quiz Application

Student Details

Student Name	M Madhumitha
Register Number	34625U50020
Department	BSc AI
ID	B40AD2D74210305AFBA45FA8DB5F2E67
College	MMES Women's Arts and Science College
Date of Submission	27 October 2026

Git-Hub Link:
https://github.com/<your-username>/online-quiz-application

1. Problem Statement
In many colleges, quizzes and practice tests are still conducted on paper. Question papers must be printed, answer sheets collected and every answer checked by hand, which takes a lot of time, wastes paper and can lead to marking mistakes. Students also have to wait a long time to know their result and cannot easily practise on their own.

Why the application is needed: A browser-based Online Quiz Application lets a student attempt a quiz at any time on a computer or phone, with no installation or login. Answers are checked automatically, feedback is shown immediately after every question, and the final score is displayed at the end. This saves time, removes manual errors and makes learning more interactive.

2. Objectives
- To develop a simple, responsive Online Quiz Application using HTML, CSS and JavaScript.
- To display multiple-choice questions one at a time with four options each.
- To check the selected answer instantly and show whether it is correct or wrong.
- To add a 20-second timer for every question to make the quiz more challenging.
- To calculate the final score and percentage and show a performance remark.
- To validate the user's name before starting and to allow the quiz to be restarted.

Expected outcome: A fully working quiz website that conducts an 8-question Web Development quiz, gives instant feedback, and displays the final result, demonstrating the fundamentals of web development.

3. Architecture
The application is a **client-side single-page web app**. All logic runs in the browser and no server or database is needed. The three screens (Start, Quiz and Result) are inside one HTML page, and JavaScript shows only one screen at a time. The project has three files:

- **index.html** – structure of the Start, Quiz and Result screens.
- **style.css** – colours, card layout, option buttons, progress bar and responsive design.

- **script.js** – question data, timer, answer checking, score calculation and screen switching.

Technologies used and their roles

Technology	Role
HTML5	Creates the structure of the three screens: heading, input box, buttons, progress bar and containers.
CSS3	Styles the application with a gradient background, card design, hover effects, and correct (green) / wrong (red) option colours; media-friendly width.
JavaScript (ES6)	Stores questions in an array of objects, handles click events, runs the timer, checks answers and shows the score.
VS Code / Browser / GitHub	Code editor, testing environment (Chrome) and version control/hosting.

User navigation flow

The user opens the page and sees the Start screen. After entering a name and clicking Start Quiz, the Quiz screen appears with one question at a time. After each answer the user clicks Next. After the last question, the Result screen shows the score, and the user can restart the quiz.

Application Flow Diagram

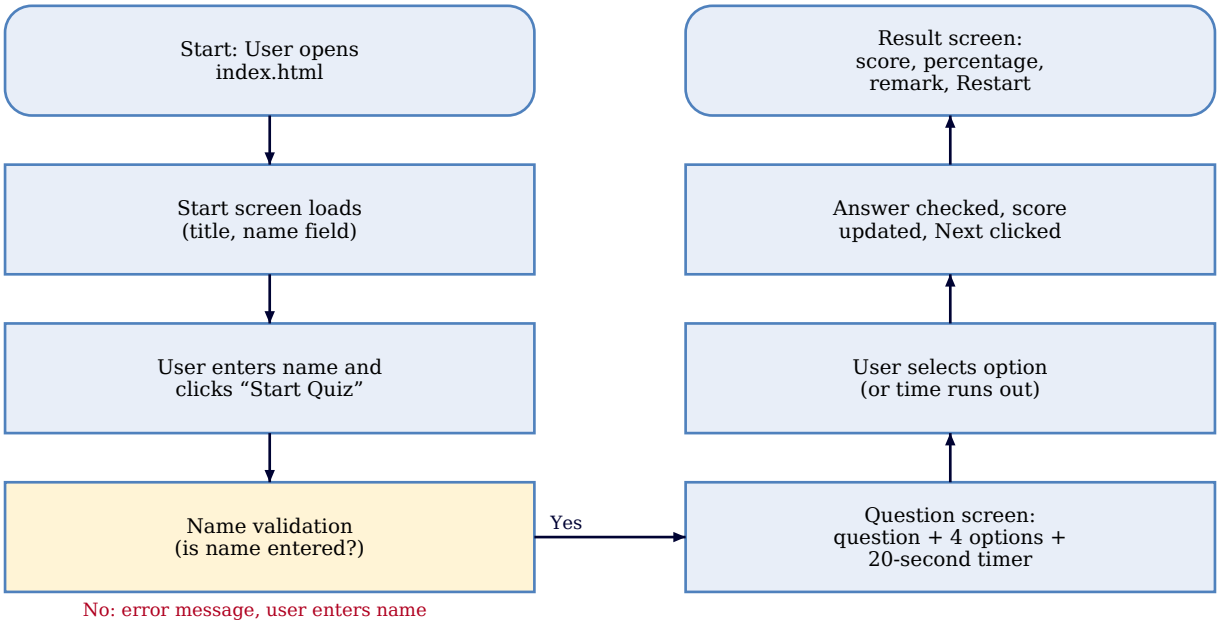


Figure 1: Application flow from the initial page to the final output

4. Practical Implementation

The application was developed in stages: (1) planning the screens and features, (2) writing the HTML for the three screens, (3) styling with CSS, (4) creating the question bank as a JavaScript array, (5) writing the functions for starting the quiz, loading questions, timer, answer checking and result, (6) testing with correct, wrong and timed-out answers and on a mobile-width screen, and (7) uploading the project to GitHub.

4.1 Features Implemented

Feature	Purpose
Name validation	Does not start the quiz until the student enters a name; shows an error message otherwise.
Multiple-choice questions	Shows 8 Web Development questions, one at a time, with four options.
Countdown timer	Gives 20 seconds per question; if time ends, the correct answer is shown automatically.
Instant feedback	Correct option turns green and a wrong choice turns red immediately.
Progress bar and counter	Shows “Question X of 8” and a bar so the user knows the progress.
Score calculation	Counts correct answers and shows score and percentage.
Result remark	Shows a message such as “Excellent work!” depending on the percentage.
Restart option	Returns to the Start screen so the quiz can be attempted again.
Responsive design	Card width adjusts to desktop and mobile screens.

4.2 Source Code

The source code of the three files is given below with an explanation of its functionality.

HTML

HTML structures the content into three section elements: **startScreen** (title, name input, error text and Start button), **quizScreen** (question counter, timer, progress bar, question text, options container and Next button) and **resultScreen** (score, remark and Restart button). The class hidden is used to hide screens that are not active. The CSS file is linked in the head and the JavaScript file is loaded at the end of the body.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Quiz Application</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Online Quiz Application</h1>

    <!-- Start Screen -->
    <section id="startScreen" class="screen">
      <h2>Web Development Quiz</h2>
      <p>8 questions | 20 seconds per question</p>
      <input type="text" id="playerName" placeholder="Enter your name">
      <p id="error"></p>
      <button id="startBtn">Start Quiz</button>
    </section>

    <!-- Quiz Screen -->
    <section id="quizScreen" class="screen hidden">
      <div class="top">
        <span id="counter"></span>
        <span id="timer"></span>
      </div>
      <div class="bar"><div id="progress"></div></div>
      <h3 id="question"></h3>
      <div id="options"></div>
      <button id="nextBtn" class="hidden">Next</button>
    </section>

    <!-- Result Screen -->
    <section id="resultScreen" class="screen hidden">
      <h2>Quiz Completed!</h2>
      <p id="resultName"></p>
      <div class="score" id="score"></div>
      <p id="remark"></p>
      <button id="restartBtn">Restart Quiz</button>
    </section>
  </div>
  <footer>Developed by M Madhumitha | MMES Women's Arts and Science College</footer>
  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS styles the application: a gradient page background, a white rounded card in the centre, Flexbox for the counter/timer row, a progress bar with smooth transition, and full-width option buttons. The classes .correct and .wrong give the green and red feedback colours, and .hidden hides elements using display:none.

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Arial, sans-serif; }
body { background: linear-gradient(135deg, #5b6df0, #8e4fd8); min-height: 100vh; display: flex; flex-direction: column; align-items: c
.container { background: #fff; width: 92%; max-width: 560px; padding: 28px; border-radius: 16px; box-shadow: 0 8px 25px rgba(0,0,0,.25
h1 { text-align: center; color: #4a4fd1; margin-bottom: 18px; }
.screen { text-align: center; }
.hidden { display: none; }
input { width: 100%; padding: 11px; margin: 14px 0 6px; border: 1px solid #bbc; border-radius: 8px; }
button { background: #4a4fd1; color: #fff; border: none; padding: 11px 24px; border-radius: 8px; cursor: pointer; font-size: 1rem; mar
button:hover { background: #3a3eb0; }
#error { color: #d9363e; min-height: 20px; font-size: .9rem; }

.top { display: flex; justify-content: space-between; font-weight: 600; color: #444; }
#timer { color: #d9363e; }
.bar { height: 8px; background: #e5e7f5; border-radius: 6px; margin: 10px 0 18px; overflow: hidden; }
#progress { height: 100%; width: 0; background: #4a4fd1; transition: width .3s; }
#question { margin-bottom: 14px; text-align: left; }
.option { display: block; width: 100%; text-align: left; background: #f1f3ff; color: #222; border: 2px solid #d6daf5; margin: 8px 0; }
.option:hover { background: #e2e6ff; }
.option.correct { background: #d4f5dd; border-color: #2e9e55; }
.option.wrong { background: #fbd9db; border-color: #d9363e; }
.score { font-size: 2.6rem; font-weight: bold; color: #4a4fd1; margin: 14px 0; }
footer { color: #fff; margin-top: 16px; font-size: .85rem; }
```

JavaScript

JavaScript implements the logic. The **questions** array stores each question, its four options and the index of the correct answer. **startQuiz()** validates the name and shows the quiz screen; **loadQuestion()** displays the question and creates option buttons; **startTimer()** uses setInterval() for the countdown; **checkAnswer()** compares the selected option with the correct one, adds the colour classes and updates the score; **nextQuestion()** moves ahead; and **showResult()** calculates the percentage and displays the final result.

```
const questions = [
  { q: "What does HTML stand for?", options: ["Hyper Text Markup Language", "High Text Machine Language", "Hyper Tool Multi Language",
  { q: "Which tag is used to create a hyperlink?", options: ["<link>", "<a>", "<href>", "<url>"], answer: 1 },
  { q: "Which CSS property changes the text colour?", options: ["font-style", "text-color", "color", "background"], answer: 2 },
  { q: "Which CSS layout system is two-dimensional?", options: ["Flexbox", "Float", "Grid", "Inline"], answer: 2 },
  { q: "Which keyword declares a constant in JavaScript?", options: ["var", "let", "const", "static"], answer: 2 },
  { q: "Which method selects an element by its id?", options: ["getElementById()", "querySelectorAll()", "getElementsByTag()", "findId
  { q: "Which symbol is used for comments in JavaScript?", options: ["<!-- -->", "//", "##", "**"], answer: 1 },
  { q: "Which HTML tag is used to link a JavaScript file?", options: ["<js>", "<link>", "<script>", "<style>"], answer: 2 }
];
const TIME_LIMIT = 20;
let current = 0, score = 0, timeLeft = TIME_LIMIT, timerId = null, answered = false, player = "";

const $ = id => document.getElementById(id);
const show = id => { ["startScreen", "quizScreen", "resultScreen"].forEach(s => $(s).classList.add("hidden")); $(id).classList.remove(

function startQuiz() {
  player = $("playerName").value.trim();
  if (player === "") { $("error").textContent = "Please enter your name to start."; return; }
  $("error").textContent = "";
  current = 0; score = 0;
  show("quizScreen");
  loadQuestion();
}

function loadQuestion() {
  answered = false;
  const item = questions[current];
  $("counter").textContent = `Question ${current + 1} of ${questions.length}`;
  $("progress").style.width = (current / questions.length) * 100 + "%";
  $("question").textContent = item.q;
  $("nextBtn").classList.add("hidden");
  $("options").innerHTML = "";
  item.options.forEach((text, i) => {
    const btn = document.createElement("button");
    btn.className = "option";
    btn.textContent = text;
    btn.onclick = () => checkAnswer(i);
    $("options").appendChild(btn);
  });
  startTimer();
}

function startTimer() {
  clearInterval(timerId);
  timeLeft = TIME_LIMIT;
  $("timer").textContent = "□ " + timeLeft + "s";
  timerId = setInterval(() => {
    timeLeft--;
    $("timer").textContent = "□ " + timeLeft + "s";
    if (timeLeft === 0) checkAnswer(-1);   // time over
  }, 1000);
}

function checkAnswer(selected) {
  if (answered) return;
  answered = true;
  clearInterval(timerId);
  const correct = questions[current].answer;
  const buttons = document.querySelectorAll(".option");
  buttons.forEach((b, i) => {
    if (i === correct) b.classList.add("correct");
    if (i === selected && selected !== correct) b.classList.add("wrong");
    b.disabled = true;
  });
  if (selected === correct) score++;
  $("nextBtn").textContent = current === questions.length - 1 ? "Show Result" : "Next";
  $("nextBtn").classList.remove("hidden");
}

function nextQuestion() {
  current++;
  current < questions.length ? loadQuestion() : showResult();
}

function showResult() {
  show("resultScreen");
  const percent = Math.round((score / questions.length) * 100);
  $("resultName").textContent = "Well done, " + player + "!";
  $("score").textContent = score + " / " + questions.length + " (" + percent + "%)";
  $("remark").textContent = percent >= 80 ? "Excellent work!" : percent >= 50 ? "Good, keep practising!" : "Keep learning and try agai
}

$("startBtn").addEventListener("click", startQuiz);
$("nextBtn").addEventListener("click", nextQuestion);
$("restartBtn").addEventListener("click", () => { $("playerName").value = ""; show("startScreen"); });
```

4.3 Project Execution

How to run:

- Download or clone the project from GitHub and keep index.html, style.css and script.js in the same folder.
- Double-click **index.html** to open it in Chrome, Edge or Firefox (or use VS Code “Live Server”).
- No installation, database or internet connection is required.

Working process: The student enters a name and clicks Start Quiz. The first question appears with four options and a 20-second timer. When the student selects an option, the timer stops, the correct answer turns green (and a wrong choice turns red), and the Next button appears. If the timer reaches zero, the correct answer is shown automatically. This repeats for all 8 questions. Finally, the Result screen shows the score, percentage and a remark, and the student can restart the quiz.

5. Output Screenshots

The following screenshots show the working application.

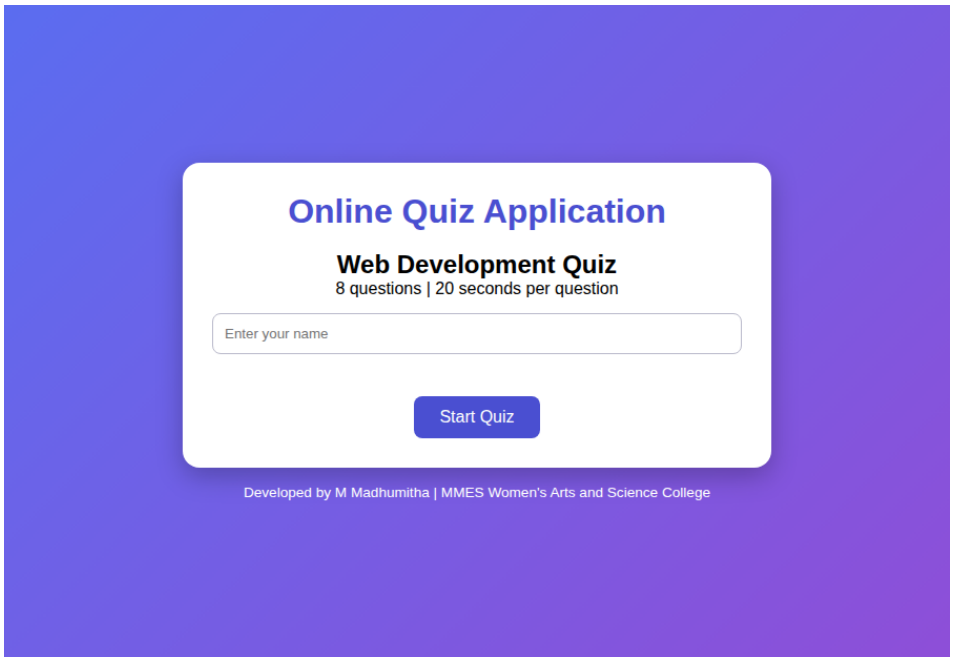


Figure 2: Start screen

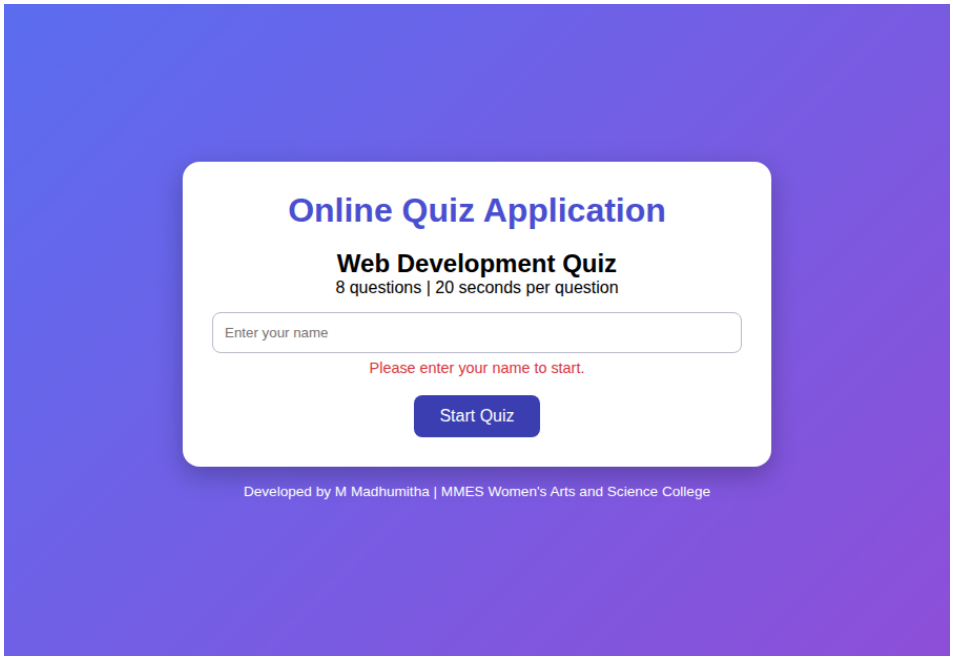


Figure 3: Name validation – error shown when the name is empty

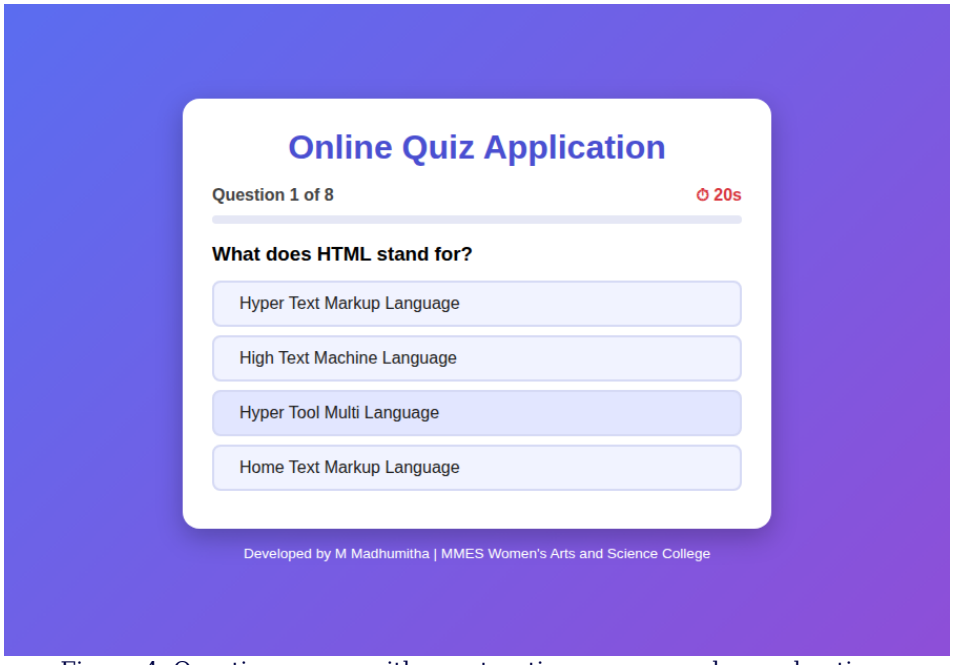


Figure 4: Question screen with counter, timer, progress bar and options

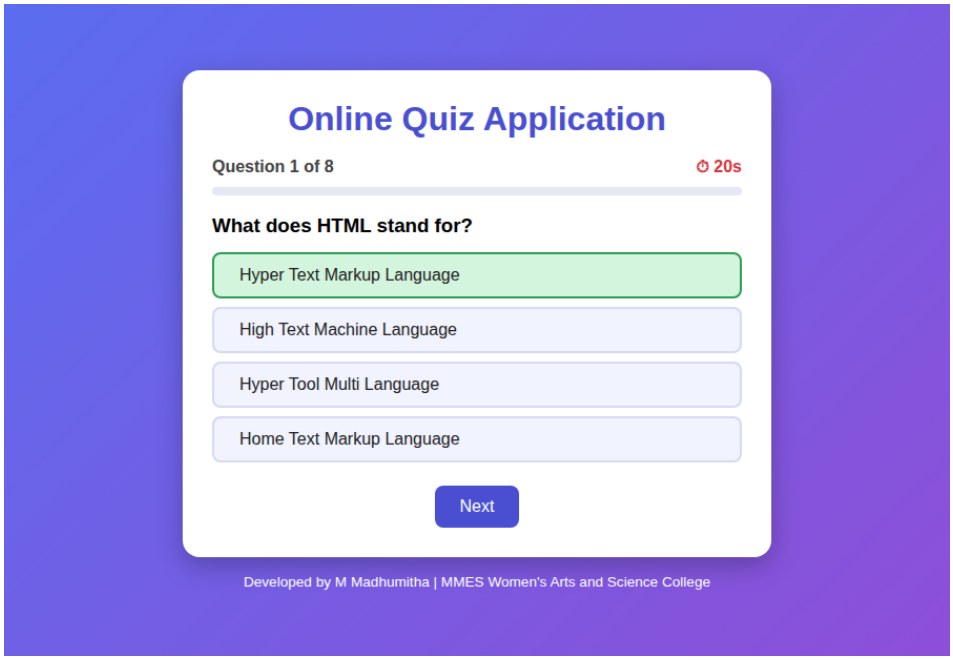


Figure 5: Correct answer selected – option turns green and Next button appears

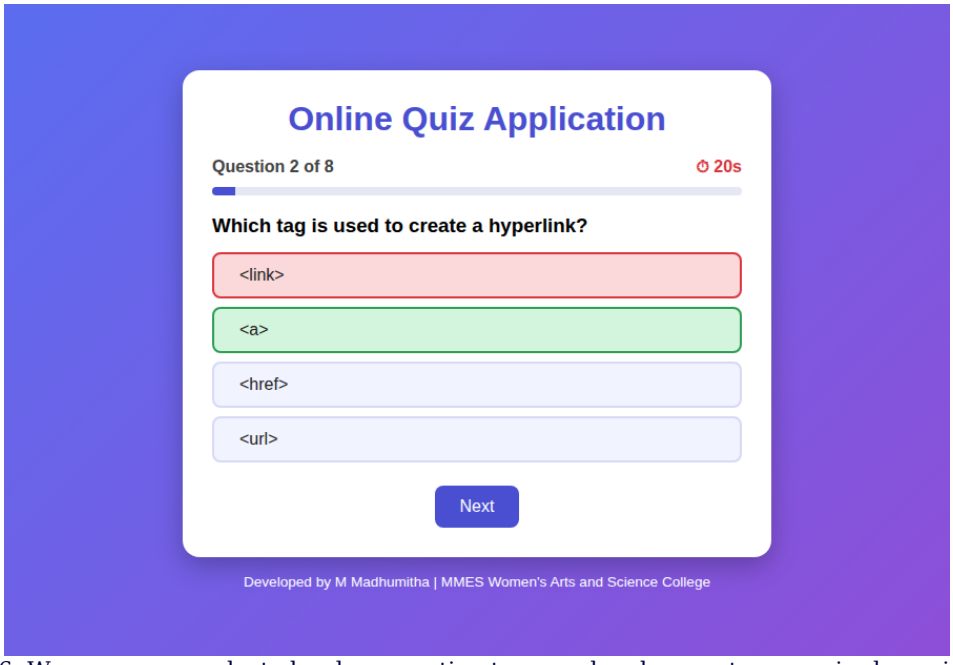


Figure 6: Wrong answer selected – chosen option turns red and correct answer is shown in green

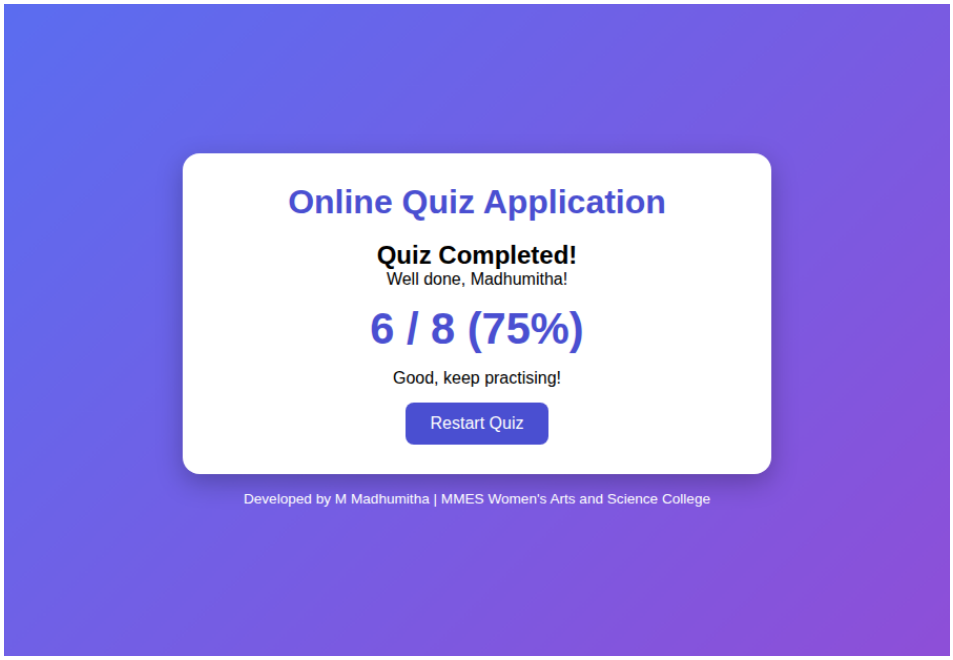


Figure 7: Result screen showing score 6 / 8 (75%) and remark

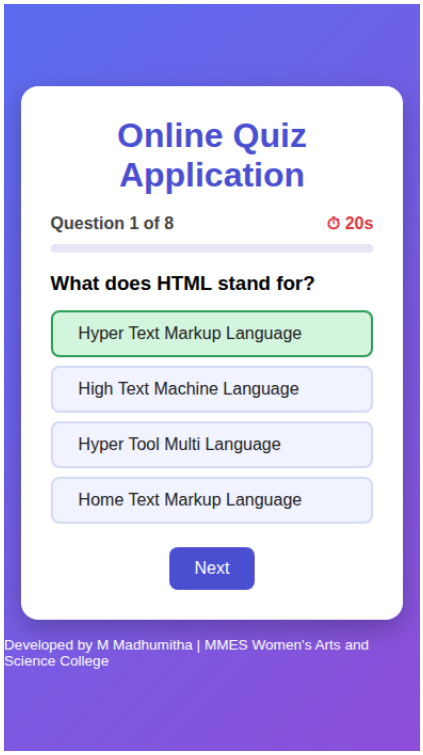


Figure 8: Mobile view (390 px width)

6. Technical Explanation

- **HTML structure and DOM:** One page contains three sections. JavaScript uses getElementById and classList to show or hide them, so the page never reloads.
- **CSS styling:** Flexbox, gradients, box-shadow, hover effects and class-based colour feedback create a clean user interface.
- **Arrays and objects:** Each question is an object {q, options, answer}; all questions are stored in an array and accessed by an index variable.
- **Event handling:** Click events on Start, option, Next and Restart buttons drive the program.
- **Dynamic DOM creation:** createElement() and forEach() generate the four option buttons for each question.
- **Timer logic:** setInterval() reduces the time every second and clearInterval() stops it when an answer is chosen or the time ends.
- **Conditional logic:** If-statements and the ternary operator validate input, compare answers and choose the result remark.
- **Score calculation:** Percentage = (score ÷ total questions) × 100, rounded using Math.round().

7. Conclusion

The Online Quiz Application was developed successfully using HTML, CSS and JavaScript. It conducts a timed multiple-choice quiz, gives instant feedback on every answer, calculates the final score and lets the user restart. The application is responsive and works in any modern browser without installation.

Through this project, I learned how to structure a single-page application, style it attractively, handle user events, work with arrays and objects, update the DOM dynamically and use timers in JavaScript. It improved my understanding of the fundamentals of web development and how HTML, CSS and JavaScript work together.

8. Future Enhancements

- **New features:** multiple quiz topics and difficulty levels, randomised questions, review of wrong answers and a leaderboard.
- **Integration:** A backend (Node.js with a database such as MongoDB) to store questions and results, and an admin page to add or edit questions.
- **Enhanced security:** student login, protection against cheating (tab-switch detection), and server-side score checking so answers cannot be seen in the browser code.
- **Improved user experience:** sound effects, animations, dark mode, Tamil/English language options, accessibility support and a certificate download for high scorers.